C200 Programming Assignment № 11

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

April 20, 2024

Introduction

In this homework, you'll work on translating critical thinking to programming. This is the **last homework**, you'll be reading some on your own and experimenting.

Make sure to push your work and submit to Autograder by **April, 27th, Saturday, 11:00 PM**

1: Numismatics

An unscrupulous collectors of coins claims to have n of a particular type. He is lying. There is one fake. You don't have time to scrutinize each coin. Using a balance you can efficiently discover the fake coin (Fig. 1).

Figure 1: Scale balance for three bags.

1. (base case) If there are only three coins x, y, z, then this returns the value:

```
(x==y)*z + (y==z)*x + (x==z)*y
```

   which, in words, says if x, y weight the same, the fake is z and so on.

2. (inductive case) separate the coins into three equal piles. There obviously might be one or two coins, since n might be divisible by three or have a remainder of 1 or 2.

```
>>> 20 % 3, 21 % 3, 22 % 3
(2, 0, 1)
```

   Two bags must have the same weight. The third bag has the fake. So you split the third bag into three piles (including the 1 or 2 extra if they exist) and repeat the process.

You'll be given a list of coins. Each real coin is worth 1. The fake coin is worth a different positive integer value. For example:

```
coins = [1,1,1,1,1,1,1,1,1,1,1,1,1,2,1]
```

You'll complete the *recursive find* that takes a list of coins and returns the fake. When you're done, we'll use $\langle list \rangle.index(x)$ method to find the index of the fake coin. The scale is called by the function weight(x,y):

```
cnt = 0

def weigh(x,y):
    global cnt
    cnt+=1
    return sum(x) == sum(y)
```

This it the only function you can use in the code unless there are *exactly* three coins. The global cnt allows us to update the global variable locally. The following code:

```
coins = [1,1,1,1,1,1,1,1,1,1,1,1,1,2,1]
print(coins)
fake = find(coins)
print(fake,coins.index(fake),cnt)
```

produces:

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1]
2 13 2
```

The fake coin is 2, its index is 13, and we used the balance scale twice. How can that be possible? We'll show the successive weighings:

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1]  →
```
$\underbrace{[1,1,1,1,1][1,1,1,1,1]}$ `[1, 1, 1, 2, 1] [] →`

$\underbrace{[1,1][1,1]}$ `[1, 2] [1] →`

```
2 13 2
```

There were two weighings. The cnt variable keeps track of the number of weighings.

Programming Problem 1: Fake Coin

- Complete the functions. The function find is recursive.
- Do not modify weigh.
- split_weight takes a none-empty list of at least 3 numbers and returns four lists, the fourth can be empty, contain one, or contain two numbers. For example,

```
print(split_weight([1,1,1,1,1,2,1,1]))
```

produces

```
([1, 1], [1, 1], [1, 2], [1, 1])
```

Problem 2: Heiroglyphics

Heiroglyphics was the system of writing in Ancient Egypt. The knowledge that it was writing was lost for many centuries. We now have some strong sense of the sounds of most of the symbols.

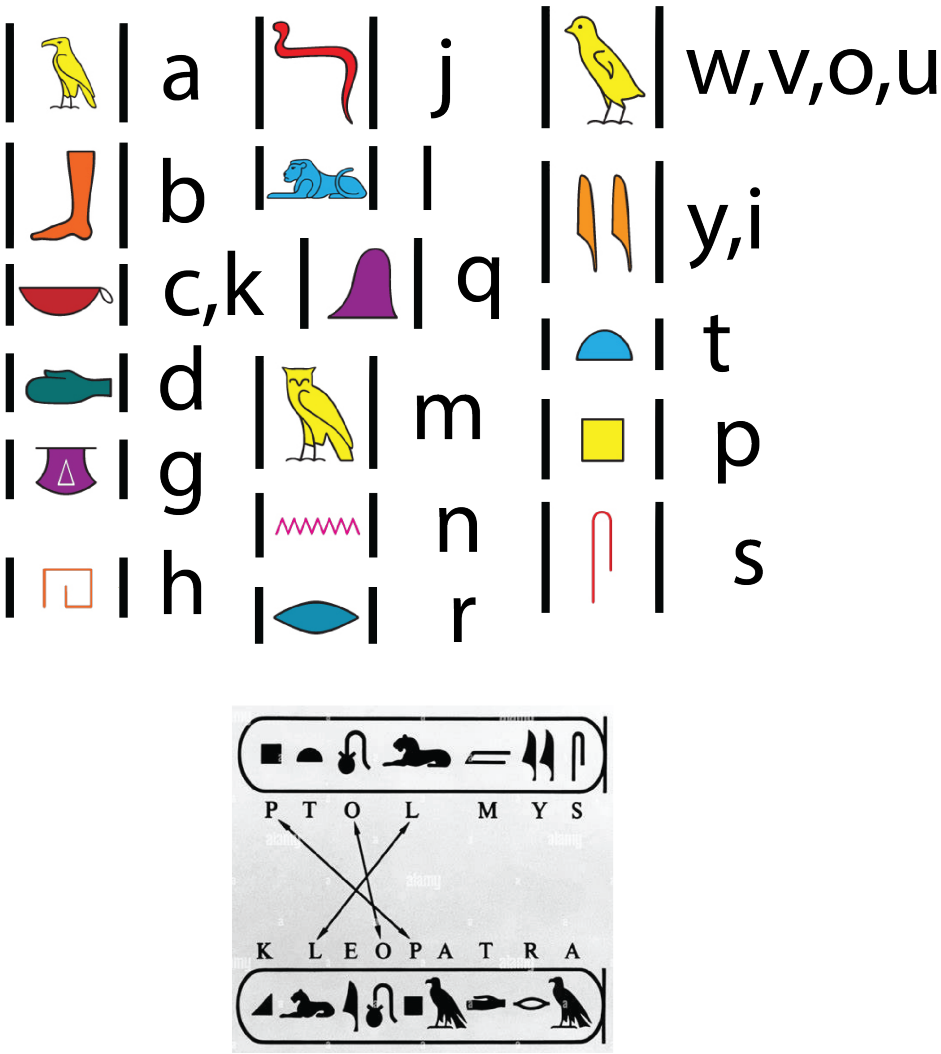

Figure 2: Heiroglyphs and approximate sounds.

Figure 3 shows one list of heiroglyphs and sounds. As you can observe, not all sounds in English have a corresponding heiroglyph. This program will read your name from a file, then translate it to the appropriate symbols and create an HTML page with the cartouche (surrounding ellipse) indicating it's a formal, important name.

Each of the symbols has a correponding jpg image with the name eglyph$x$, where x is the letter. The top and bottom of the cartouche are eglyphTOP.jpg, eglyphBOTTOM.jpg, respectively. You'll finish a class for building HTML pages and a couple of function. The following code:

```
page = my_HTML()
print(page)
page.add_element(f"{name} = {translate(name)}", element = 'h2' )
print(page)
```

```
5 page = page + "eglyphTOP.jpg"
6 add_hieroglyphs(page,name)
7 page = page + "eglyphBOTTOM.jpg"
8 print(page)
9 page.save(translate(name))
```

The HTML is nothing more than a simple string.

- Line 1 creates an instance of the class.
- Line 2 prints the current page. This is a blank page (look at the output).
- Line 3 adds the H2 header. Which displays the first line of the webpage.
- Line 4 prints it.
- Line 5 uses an overloaded + to add an image. This adds the string:

```
1 <img src='eglyphTOP.jpg' class='center'><br>
```

given a string that describes the file.

- Line 6 is a function that takes a webpage and a name and adds the image tags as filenames to the page.
- Line 9 is a method that takes a string and writes the HTML (string) to that location.

adds h2 element. As you can see we're adding the name = translate(name) as a string. We're using a switch to indicate this must be the h2 tag. produces:

```
1 <html><head></head><body></body></html>
2
3 <html><head></head><body><h2 class='text-center'> Mehmet = mhmt </h2>
4 </body></html>
5
6 <html><head></head><body><h2  class='text-center'> Mehmet = mhmt </h2>
7 <img src='eglyphTOP.jpg' class='center'><br>
8 <img src='eglyphm.jpg' class='center'><br>
9 <img src='eglyphh.jpg' class='center'><br>
10 <img src='eglyphm.jpg' class='center'><br>
11 <img src='eglypht.jpg' class='center'><br>
12 <img src='eglyphBOTTOM.jpg' class='center'><br>
13 </body></html>
```

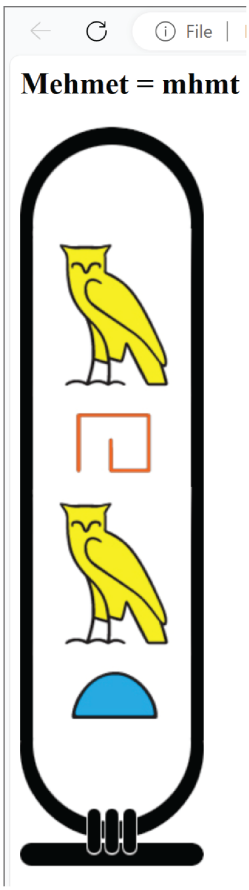

Figure 3: Heiroglyphs and approximate sounds.

Deliverable for Problem 2: The Mummy

- Complete the class. I've finished the init class and string class.
- Observe that some letters in a name will be ignored, since there isn't any corresponding heiroglyph.
- The translate function should change to lower case.
- All the files are given (pushed to your repo) and you should look at them.
- The HTML should be saved by your Egyptian name as the Egyptianname.html.
- I'll include the file I created (called mhmt.html). You are welcome to look at it.

Problem 3: Earth

As your last problem in C200, I thought having you render NASA's Blue Marble image would fun. I hope you'll see how easy and useful libraries are. Figure 4 shows the output of Earth from some distance away.

Figure 4: Picture of Earth.

Here's the code:

```
# setup Lambert Conformal basemap.
# set resolution=None to skip processing of boundary datasets.
m = Basemap(width=12000000,height=9000000,projection='lcc',
            resolution=None,lat_1=45.,lat_2=55,lat_0=50,lon_0=-107.)
m.bluemarble()
plt.show()
```

The relative page is `https://matplotlib.org/basemap/stable/users/geography.html`.

Decision Tree

- Get the code to run.
- You'll likely have to install basemap (the installation is on the website).

1 Programming Partners

pkurzawa@iu.edu, camlueck@iu.edu
belleli@iu.edu, ap59@iu.edu
nbkotur@iu.edu, ptapiaro@iu.edu

zcarmich@iu.edu, wangjinz@iu.edu
blberger@iu.edu, weinstab@iu.edu
blevidut@iu.edu, leoett@iu.edu
anchinna@iu.edu, bfmark@iu.edu
jvgottum@iu.edu, nicmill@iu.edu
burkhel@iu.edu, ksreejit@iu.edu
balian@iu.edu, benserna@iu.edu
rachguo@iu.edu, kammich@iu.edu
jdemian@iu.edu, corrosar@iu.edu
chamblia@iu.edu, joebrahm@iu.edu
shbivens@iu.edu, katepate@iu.edu
forlin@iu.edu, traepena@iu.edu
zlangley@iu.edu, tmukund@iu.edu
amgandhi@iu.edu, peterant@iu.edu
marbelli@iu.edu, mgstack@iu.edu
jakedeck@iu.edu, nsenthi@iu.edu
nabatja@iu.edu, wagnebr@iu.edu
allentp@iu.edu, arangi@iu.edu
lasdodd@iu.edu, bsai@iu.edu
harrisdg@iu.edu, javitto@iu.edu
camehugh@iu.edu, ntorpoco@iu.edu
askaul@iu.edu, ctsvetan@iu.edu
jailjohn@iu.edu, njvinson@iu.edu
ankotel@iu.edu, jianlou@iu.edu
hedavid@iu.edu, amatariy@iu.edu
tkonkim@iu.edu, reddysk@iu.edu
anudivek@iu.edu, brshelby@iu.edu
lrborows@iu.edu, arstover@iu.edu
pafrost@iu.edu, andmuchi@iu.edu
ml82@iu.edu, lawlin@iu.edu
mgrass@iu.edu, johnsanc@iu.edu
njustin@iu.edu, ashahi@iu.edu
samjimen@iu.edu, animerch@iu.edu
cdinman@iu.edu, etmillho@iu.edu
jhbruce@iu.edu, jmatczak@iu.edu
ecabebe@iu.edu, tiffyou@iu.edu
chenkay@iu.edu, rimaeda@iu.edu
chodgman@iu.edu, ishetty@iu.edu
markdeng@iu.edu, jusshah@iu.edu
ryandaun@iu.edu, trimedio@iu.edu
ejbusch@iu.edu, nmetlapa@iu.edu

adeglur@iu.edu, shmulla@iu.edu
kespejo@iu.edu, wardjohn@iu.edu
tkalale@iu.edu, jlopezmo@iu.edu
renahan@iu.edu, surehma@iu.edu
jamlian@iu.edu, syeshala@iu.edu
cc96@iu.edu, gravipat@iu.edu
sambereb@iu.edu, frvisco@iu.edu
ecanant@iu.edu, iopfings@iu.edu
arkhal@iu.edu, cmoverfe@iu.edu
conkeen@iu.edu, twirges@iu.edu
harmonnc@iu.edu, aquintas@iu.edu
langgrav@iu.edu, bpovinel@iu.edu
allennam@iu.edu, rsreedh@iu.edu
cgreywal@iu.edu, ymarrakc@iu.edu
achitred@iu.edu, nsmurthy@iu.edu
akumares@iu.edu, camyee@iu.edu
berrynm@iu.edu, teswank@iu.edu
nkushnir@iu.edu, abbalokh@iu.edu
lineri@iu.edu, avaweish@iu.edu
craftdc@iu.edu, khwillar@iu.edu
changali@iu.edu, pratmodi@iu.edu
sdharmmi@iu.edu, olipfing@iu.edu
nsfried@iu.edu, ggriorda@iu.edu
ocrowder@iu.edu, jocpoiri@iu.edu
daveanu@iu.edu, haoztian@iu.edu
reychaud@iu.edu, xsoto@iu.edu
cbremner@iu.edu, shubtiwa@iu.edu
cedonica@iu.edu, mxrobins@iu.edu
nkhai@iu.edu, msolomie@iu.edu
logellis@iu.edu, nvoneper@iu.edu
presharr@iu.edu, zstopher@iu.edu
okorkmaz@iu.edu, menonvi@iu.edu
emiellis@iu.edu, lomwean@iu.edu
wchorich@iu.edu, colbstap@iu.edu
sjbarry@iu.edu, rahkpate@iu.edu
sgudapur@iu.edu, joprayit@iu.edu
jkchaura@iu.edu, vupatel@iu.edu
kimyouh@iu.edu, smremmer@iu.edu
madimond@iu.edu, masoroki@iu.edu
rcerauli@iu.edu, adpulipa@iu.edu
gmhowell@iu.edu, peterabl@iu.edu

boding@iu.edu, gstogdil@iu.edu
aadkhan@iu.edu, snlnu@iu.edu
beckab@iu.edu, alexpan@iu.edu
adevill@iu.edu, jamread@iu.edu
arkali@iu.edu, sparab@iu.edu
reafrank@iu.edu, breramer@iu.edu
zaycarey@iu.edu, ceprenti@iu.edu
bcabral@iu.edu, jomalek@iu.edu
shaagore@iu.edu, tryerrag@iu.edu
cfahy@iu.edu, mvsimi@iu.edu
ykassahu@iu.edu, mitcchar@iu.edu
andhodge@iu.edu, archshar@iu.edu
abelani@iu.edu, smunikun@iu.edu
aydclink@iu.edu, auspiege@iu.edu
sebcont@iu.edu, bwmoore@iu.edu
auconner@iu.edu, saravula@iu.edu
cjduffie@iu.edu, crschuit@iu.edu
careyjo@iu.edu, amtsmith@iu.edu
agunde@iu.edu, nrimawi@iu.edu
mbadwe@iu.edu, cmm23@iu.edu
jaxkatz@iu.edu, mackeyr@iu.edu